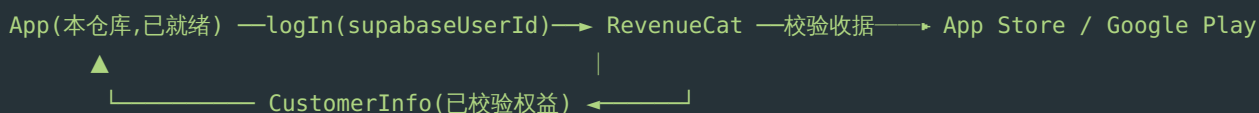


支付订阅接入与 **Sandbox** 测试指南 (**RevenueCat** 标准链路)

代码侧已完成：RevenueCat SDK 集成、Paywall、订阅入口、权益状态机。要真实跑通购买，必须先完成下面三处后台配置 + 注入 **API Key**。跳过配置 SDK 会拉不到商品、无法购买——这是标准链路里最常见的坑。

0. 总览：标准链路三方



App 只认 RevenueCat 返回的、已服务端校验的 **CustomerInfo**，不自行验收据。退款/续费由 RevenueCat 处理后通过 CustomerInfo 同步回来。

1. App Store Connect (iOS)

1. 同意 **Paid Apps** 协议：Business → Agreements，必须是 Active，否则商品不可用。
2. 创建订阅商品：App → 订阅 → 新建订阅组（Echo Loop Plus）→ 组内建两个订阅：
3. 月订：Product ID `echo_loop_plus_monthly`
4. 年订：Product ID `echo_loop_plus_annual`
5. 年订加免费试用：年订 → 推介促销优惠（Introductory Offer）→ Free trial 7 天。
6. 填本地化信息 + 价格：每个商品的显示名、描述、各区价格（含中国大陆区，注意大陆自动续费披露要求）。
7. **App** 内购买密钥（**In-App Purchase Key**）：用户与 App 信息 → App 内购买 → 生成密钥，供 RevenueCat StoreKit 2 用。
8. **Sandbox** 测试员：用户与访问 → Sandbox → 测试员，新建一个测试 Apple ID（用一个没在真实 App Store 登录过的邮箱）。

2. Google Play Console (Android)

1. 创建订阅：营收 → 订阅 → 新建订阅：
2. `premium_monthly`（base plan：每月，自动续费）
3. `premium_yearly`（base plan：每年，自动续费；offer：7 天免费试用）
4. 激活商品，并上传一个含本订阅 SDK 的构建到「内部测试」轨道（订阅在有上架构建后才生效）。

5. **License testers** : 设置 → 许可测试 → 加入测试 Google 账号 (沙盒扣款不真实计费)。
6. **Service Account** : 建一个 GCP service account , 授予 Play Developer API 权限, 下载 JSON , 供 RevenueCat 用。

3. RevenueCat 后台

1. 新建 Project , 加两个 App :
2. iOS App : 填 bundle id + 上传 App 内购买密钥 (.p8) 。
3. Android App : 填 package name + 上传 service account JSON。
4. **Entitlement** : Entitlements → 标识为 Echo Loop Plus 的 entitlement (必须与代码里 REVENUECAT_ENTITLEMENT_ID 一致, 默认 Echo Loop Plus) 。
5. **Products** : 把上面 4 个商店商品 (iOS 月/年 + Android 月/年) 导入, 并都 **attach** 到 Echo Loop Plus entitlement。
6. **Offering** : 建一个 current Offering (如 default) , 加两个 Package :
 7. \$rc_monthly → 月订商品
 8. \$rc_annual → 年订商品 (代码按 packageType 区分月/年, 按 \$rc_annual 识别年付主推 + 试用。)
9. **API Keys** : Project settings → API keys , 复制两个公开 SDK key :
10. Apple public key
11. Google public key

4. 注入 API Key (本仓库)

代码通过 `--dart-define` 读取 (与 Supabase 一致)。在各环境的 `auth.env` 追加 :

```
REVENUECAT_API_KEY_APPLE=appl_XXXXXXXXXXXXXXXXXX
REVENUECAT_API_KEY_GOOGLE=goog_XXXXXXXXXXXXXXXXXX
# 可选, 默认 Echo Loop Plus, 需与 RevenueCat entitlement 标识一致
REVENUECAT_ENTITLEMENT_ID=Echo Loop Plus
```

运行 :

```
flutter run --dart-define-from-file=auth.env
```

平台启停语义 (编译期开关)

订阅实现由 `platform + DISTRIBUTION_CHANNEL` 本地决定 :

- `ios|macos + app_store` → Apple IAP / StoreKit ; `android + play` → Google Play / RevenueCat Google。
- `android|macos|windows + direct` → RevenueCat Web Purchase Link / Paddle。
- 未注入 key 的平台 : RC 不初始化、**订阅入口整体隐藏** (设置页 tile 不渲染、`openPaywall` 拦截并提示、Paywall 路由显示占位页) , app 照常匿名运行。

- 未注入或非法 distribution 保守隐藏支付入口；旧值 `apk / desktop` 仅在客户端解析层兼容为 `direct`，不再作为正式 header 值。
- 停用/启用支付需要发版；**限额侧按平台+渠道组合由后端 `env` 控制**（见第 7 节 `AI_QUOTA_ENFORCED_CLIENTS`）。

相关代码：`lib/config/revenuecat_config.dart`（读 `key` + `isSubscriptionSupported`）、`lib/features/subscription/providers/subscription_availability.dart`（UI 门控 `provider`）、`lib/main.dart`（`Purchases.configure`）。

非商店渠道：RevenueCat Web Paywall + Paddle

Android 侧载 APK / 桌面等非商店渠道不初始化 RevenueCat 原生 SDK，购买入口打开 RevenueCat 托管 Web Paywall（底层 Paddle 计费），权益仍经 RevenueCat webhook 落库后由 `/api/entitlements` 读回。

构建时注入：

```
DISTRIBUTION_CHANNEL=direct
WEB_PURCHASE_LINK_TEMPLATE=https://pay.rev.cat/<token>/{app_user_id}/paywall
```

`{app_user_id}` 会被客户端替换为 URL-encoded 的 Supabase `user.id`。模板未配置或缺 `{app_user_id}` 时，网页支付入口不可用。

Web/Paddle 的价格、五折优惠码、节日促销文案在 RevenueCat 托管 Paywall / Paddle Checkout 中维护；客户端不读取 Paddle discount，也不硬编码 Web 价格。

5. 用 Sandbox 测试账号验证

iOS（真机推荐，模拟器对 **StoreKit** 支持有限）：1. 设备 设置 → App Store → 退出真实 Apple ID（沙盒购买时系统会单独提示用 Sandbox 账号登录）。2. `flutter run --dart-define-from-file=auth.env`。3. App 内先登录 Supabase（购买前强制登录）。4. 设置页「升级 Premium」→ Paywall → 选套餐 → 订阅 → 用 **Sandbox 测试 Apple ID** 完成购买。5. 验证：购买后立即解锁（顶部变「你已是 Premium 会员」）；杀进程重启仍是会员；点「恢复购买」可恢复；到 沙盒订阅会加速过期，可验证过期降级。

Android：1. 用加入了 License testers 的 Google 账号登录设备。2. 装内部测试轨道的构建（或 `flutter run`，但商品需已在 Play 激活）。3. 同样流程购买，测试卡用 Google 测试卡，不真实计费。

5.5 本地 StoreKit 测试模式（开发用，绕过 RevenueCat）

仅想快速验证购买流程 / Paywall，又不想污染 **RevenueCat** 的 **Sandbox customers**、不想被「RC 缓存让 app 一直显示已订阅」困扰时，用本模式。原理：RC SDK 一旦 `configure` 就会捕获所有 StoreKit 交易（含 `.storekit` 本地交易）上报，无法「选择性不上报」——所以本模式下**根本不初始化 RevenueCat**，购买改走 `in_app_purchase` 直连 Xcode `.storekit`，权益状态只存在于 StoreKit 本地，单一可控。

用法：1. Xcode scheme 已挂 `.storekit` 配置文件（Edit Scheme → Run → Options → StoreKit Configuration → `EchoLoop.storekit`）。2. 加 `--dart-define=USE_LOCAL_STOREKIT=true` 启动：`bash`

`flutter run --dart-define=USE_LOCAL_STOREKIT=true` 3. 设置页 → Paywall → 购买，弹出的是 Xcode 本地模拟购买框，**不经过 Apple，也不经过 RevenueCat**。

重置状态（三方同步的关键）：状态只在 StoreKit 本地，删交易即降级——Xcode → Debug ▶ StoreKit ▶ Manage Transactions → 删除对应交易。无需清 RC、无需卸载重装。

注意：- 本模式只测「购买 UI / Paywall 门禁 / 权益状态流转」，**不验证 RC → entitlement 端到端映射**；后者必须走第 5 节真实 Sandbox 账号。- 只测会员 UI、连购买都不想发起时，用 App 内「开发者选项 → 订阅调试 → 手动覆盖权益」更省事（连 `.storekit` 都不用挂）。- release 构建禁止注入 `USE_LOCAL_STOREKIT`。

6. 验收清单（合规 + 功能）

- [] Paywall 展示：套餐名/时长/本地化价格/试用说明/自动续费披露/恢复购买/条款/隐私链接（已在代码内）。
- [] 购买成功立即解锁，重启保持。
- [] 恢复购买可用（iOS 强制要求）。
- [] 「管理订阅」跳转平台订阅页（不在 App 内取消）。
- [] 过期 / 退款后降级（沙盒可加速验证）。
- [] 跨设备：同 Supabase 账号在另一设备登录后即为会员。
- [] 大陆区自动续费披露文案符合 App Store 3.1.2。

7. 后端订阅豁免（RevenueCat → Supabase 同步）

免费额度按「用户+功能+自然月」在后端裁决（各 5 次，超额 402）。订阅用户必须在额度校验前放行，否则会被限到 5 次。这一步的代码已在后端仓库 `fluency-frontend` 实现（RC webhook → `user_entitlements` 表 → `isFreeQuotaExceeded` 放行），剩下是你需要执行的配置与上线步骤。

总开关：`AI_QUOTA_ENFORCEMENT_ENABLED`（默认关闭）

免费额度限制由后端环境变量总开关控制，**默认关闭**——不设或非 `'true'` 时，`isFreeQuotaExceeded` 一律放行（不限额、不查库），所有用户无限使用 AI 功能。订阅功能铺开、老用户过渡完成后，把它设为 `'true'` 才启用限额。

```
# 后端部署环境（Vercel 等）
AI_QUOTA_ENFORCEMENT_ENABLED=true    # 启用限额（订阅豁免 + 免费 5 次/月）
# 不设 / =false / 其它值 → 关闭（现状：所有人无限）
```

只认字面量 `'true'`；`'false'`、空、未设置都视为关闭。改这个变量后需重新部署（Vercel 环境变量在下次部署生效）。开关关闭时，webhook 仍可照常落库订阅状态（不影响），只是暂不据此限额。

平台+渠道灰度： `AI_QUOTA_ENFORCED_CLIENTS` （默认空 = 全放行）

限额按合法的 `platform:distribution` 组合灰度启用，支付尚未上线的组合继续无限用。

```
# 后端部署环境 (Vercel 等)
AI_QUOTA_ENFORCED_CLIENTS=ios:app_store,android:play
# 可选追加 macos:app_store / macos:direct / android:direct / windows:direct
# 不设 / 空 → 不对任何组合限额
```

- 合法组合： `android:play`、`android:direct`、`ios:app_store`、`macos:app_store`、`macos:direct`、`windows:direct`。
- 客户端请求同时携带 `x-app-platform` 与 `x-app-distribution`；正式 `distribution` 只允许 `play` / `app_store` / `direct`。
- 缺失、非法值或非法组合一律 **fail-open**：旧客户端缺 `distribution` 时不启用限额。
- 判定顺序（`isFreeQuotaExceeded`）：总开关 → 组合名单 → 订阅豁免 → 免费额度比较。

已实现（后端代码， `fluency-frontend`）

- `user_entitlements` 表（迁移 `0037_fluffy_william_stryker.sql`）：每（用户，**entitlement**）一行（复合唯一键，对齐 RC 官方 Supabase 集成做法，支持多权益），`is_active` + `expires_at_ms` (bigint) + `raw` (jsonb 事件原文，逃生舱) 等。
- RC webhook： `POST /api/revenuecat/webhook`，`Authorization` 共享密钥校验 → 按 `entitlement_ids` 逐行 `upsert`（购买/续费/取消/过期/退款/转移）。幂等 + 抗乱序/并发（DB 层原子 `setWhere` 丢弃旧事件）+ 错误区分（FK 违约 200 不重试；瞬时故障 500 让 RC 重投，不丢事件）。
- `hasActiveEntitlement(userId)`：用户任一行 `is_active && (expires_at_ms 为空 || > now)`。
- `isFreeQuotaExceeded` 开头调它 → 订阅用户直接放行（5 个 AI 端点零改动全覆盖）。

你需要做的（按顺序）

① 生成 **webhook** 鉴权密钥并注入后端环境变量

```
openssl rand -hex 32 # 生成一串随机值作为 Authorization 头
```

在后端部署环境（Vercel 等）加环境变量：

```
REVENUECAT_WEBHOOK_AUTH=<上面生成的随机串>
```

未配置时 `webhook` 端点拒绝所有请求（500），其余路由不受影响。

② 应用数据库迁移到线上 **Supabase**（需线上凭据）

```
cd fluency-frontend/packages/database
pnpm db:migrate # 或 pnpm db:migrate:prod (用 .env.production)
```

只建 `user_entitlements` 表 + FK + 索引，不动其它表。

③ **RevenueCat** 后台配置 **Webhook** (Project settings → Integrations → Webhooks) - **URL** : `https://<你的后端域名>/api/revenuecat/webhook` - **Authorization header** : 填 ① 生成的同一串值 (RC 会原样带在请求头, 后端逐字比对) - **Environment** : Sandbox + Production 都发 (后端按 `environment` 字段存, 不区分拦截)

④ **验证** - Sandbox 账号买一单 → 看后端日志 `[RevenueCat webhook] INITIAL_PURCHASE → applied` ; 查 `user_entitlements` 有该 user 的 `is_active=true` 行。 - 用该账号连点某 AI 功能超过 5 次 → 不再返回 402 (订阅豁免生效)。 - 取消/退款/过期后 → 该用户回到免费额度限制。

前提 : 客户端购买前强制登录, `Purchases.logIn(supabaseUserId)` 使 `RC app_user_id = Supabase auth.users.id` (webhook 据此定位用户)。匿名 id 的事件会被跳过。